

Block Vote: A Gasless, Multi-Channel Blockchain E-Voting System with Liveness-Gated Remote Voting and Polling-Station Override

Deven Wagh

Department of Computer Engineering

Institution Name

City, India

Abstract—Remote electronic voting promises accessibility but struggles with three recurring problems: voters must be identified without exposing their ballot, coercion in uncontrolled environments, and the cost and complexity that blockchain-based designs push onto voters. We present *Block Vote*, a deployed system that combines (i) an upgradeable Ethereum smart contract (VotingV3, UUPS proxy, 2-of-3 guardian governance) that records ballots under pseudonymous nullifiers and emits only salted ballot commitments; (ii) a gasless relay so that voters need no wallet, cryptocurrency, or blockchain knowledge; (iii) a two-factor voter authentication combining e-mail one-time passwords with cloud face-liveness checks and a 240° device-rotation surround scan on Android; and (iv) a multi-channel ballot policy in which a voter may cast one remote vote and change it once, while a vote cast at an administrator-activated polling station is final and overrides any remote vote, an operational coercion mitigation inspired by Estonian Internet voting. We describe the architecture, protocols, and implementation ($\approx 21,600$ lines of Solidity, JavaScript and Kotlin), and evaluate it with 190 automated tests including 30 adversarial scenarios, gas benchmarks cross-validated against 114 live Sepolia transactions, and latency measurements of the production deployment. A first ballot costs 89,946 gas (145,643 gas per voter including registration), i.e. $\approx 1.6 \times 10^{-4}$ ETH at the observed median Sepolia gas price of 1.09 gwei; privacy-preserving commitments and bounded re-voting add 34.7% over a one-shot baseline. We compare Block Vote with paper-based EVMs, Estonian i-Voting, Helios, Voatz and the Open Vote Network, and discuss the remaining trust in the relay operator.

Index Terms—blockchain, electronic voting, smart contracts, Ethereum, meta-transactions, liveness detection, coercion resistance, re-voting, UUPS proxy

I. INTRODUCTION

Elections for universities, cooperatives, professional bodies and local organisations increasingly move online, yet the dominant tools are either conventional web forms, whose integrity rests entirely on the operator, or research systems that ask voters to manage keys and pay transaction fees. Public blockchains [1], [2] offer an append-only, publicly auditable record, but naive designs leak ballots in public events, require every voter to hold a funded wallet, and give no answer to coercion in the home [3].

Block Vote targets organisational elections with a known electoral roll (an e-mail address per voter) and aims for four practical properties: *eligibility* (only enrolled, physically present humans vote), *uniqueness* (one counted ballot per

voter), *public integrity* (tallies are computed by a smart contract and every ballot is an on-chain transaction), and *usability* (no wallet, no fees, a native Android app plus an in-person fallback).

Contributions. This paper makes the following contributions:

- 1) A deployed architecture that couples a gasless relay with an upgradeable voting contract whose vote events carry only salted commitments, never plaintext choices (Section IV).
- 2) A *multi-channel ballot policy*: at most two remote ballots (first vote and one change) and a final, overriding polling-station ballot, enforced by an atomic off-chain ledger and reflected on-chain through latest-vote-counts semantics (Section V-C).
- 3) A liveness pipeline combining cloud face analysis with a 240° device-rotation surround scan that detects additional persons during capture (Section V-A).
- 4) An evaluation with 190 automated tests, reproducible gas benchmarks cross-validated on a public testnet, operational cost projections and production latency (Section VII), and a structured comparison with five existing systems (Section VIII).

II. BACKGROUND AND RELATED WORK

Supervised electronic voting. India’s standalone Electronic Voting Machines with Voter-Verifiable Paper Audit Trail (VVPAT) provide in-booth secrecy and a paper audit record but require physical presence [4].

Internet voting. Estonia has run binding national Internet voting since 2005 using national ID cards [5]. Two of its design decisions directly inform ours: voters may *re-vote* online, and a ballot cast in person supersedes an Internet ballot, which weakens the value of coercing a voter at home. Security analyses have nevertheless highlighted its reliance on trusted server-side components [6].

End-to-end verifiable systems. Helios [7] encrypts ballots in the browser, publishes them on a bulletin board and tallies homomorphically, with Benaloh-style cast-as-intended challenges [8]; its author recommends it for low-coercion settings. Receipt-based schemes [9] and coercion-resistant protocols

based on fake credentials [10] give stronger guarantees at a considerable usability cost.

Blockchain voting. The Open Vote Network [11] implements self-tallying boardroom voting on Ethereum with maximal ballot secrecy, but every voter needs an Ethereum account and pays gas, and the protocol is limited to small electorates. Numerous proposals use permissioned or public chains for e-voting [12], [13]. The Voatz mobile application, which combined biometrics with a permissioned blockchain, was shown to have serious vulnerabilities in its client and server design [14], and Park *et al.* [3] argue that blockchains do not by themselves solve the core problems of Internet voting. We take these critiques as design constraints: the blockchain in Block Vote provides public integrity of recorded ballots and tallies, while eligibility, liveness and coercion mitigation are handled explicitly by other mechanisms whose trust assumptions we state.

Upgradeability and signatures. The contract follows the Universal Upgradeable Proxy Standard (UUPS, EIP-1822) [15]; privileged dashboard actions are authorised with EIP-191 personal signatures [16]. Liveness terminology follows ISO/IEC 30107 [17], and OTP handling follows NIST SP 800-63B guidance [18].

III. THREAT MODEL AND DESIGN GOALS

Actors. *Voters* (enrolled e-mail addresses); *election administrators* who create elections, candidates and rolls; three *guardians* who approve go-live and contract upgrades; the *relay operator* who runs the application tier and the funded relay wallet; and external *observers* who read the chain.

Adversaries. We consider (A1) ineligible or double voters, (A2) coercers who observe or pressure a voter at home, (A3) a malicious administrator, (A4) a partially compromised backend database, (A5) attacks on contract logic, (A6) network and relay-level attacks, and (A8) collusion among guardians. These map one-to-one onto our adversarial test suite (Table V).

Trust assumptions. The relay operator is trusted not to *cancel* or *alter* submitted ballots and learns the plaintext candidate at submission time; the identity secret used to derive nullifiers is held only server-side; guardians are honest-majority. We do not claim end-to-end verifiability or cryptographic coercion resistance; Section IX discusses these gaps.

IV. SYSTEM ARCHITECTURE

Fig. 1 shows the three tiers. *Clients* are the Android app (Kotlin, Jetpack Compose, CameraX, ML Kit), a web ballot that only opens on polling-station computers activated by an administrator, and web dashboards for administrators and guardians. The *application tier* is a Next.js 15 App Router service deployed on Vercel that implements identity, liveness, vote rules, the vote ledger, the gasless relay and authorisation of privileged operations. *External services* are MongoDB Atlas (roll, elections, OTP hashes, relay log), AWS Rekognition (face analysis), Resend (e-mail) and the Ethereum Sepolia test network hosting the VotingV3 proxy.

TABLE I
DESIGN GOALS AND THE MECHANISMS THAT REALISE THEM

Goal	Mechanism in Block Vote
Eligibility	Admin-uploaded roll; e-mail OTP (bcrypt, 5 min, 3 attempts); face liveness; on-chain registration of nullifier
Uniqueness	One nullifier per e-mail; latest-vote-counts tally; atomic vote-slot ledger
Public integrity	Every ballot is a relay transaction to VotingV3; tallies maintained by contract; audit endpoint verifies tx inclusion
Ballot privacy (public)	No PII on chain; <code>VoteCastPrivate</code> emits $H(e \parallel c \parallel n \parallel s)$ only; receipts omit candidate
Coercion mitigation	One remote change of mind; final polling-station override; voting-hours window
Usability	No wallet, no fees (gasless relay); native Android app; web ballot at polling stations
Governance	2-of-3 guardian UUPS upgrades; signed guardian actions (5 min TTL)

A. Smart contract

VotingV3 (Solidity 0.8.22, OpenZeppelin upgradeable v5) is deployed behind a UUPS proxy. Its state comprises elections (title, description, banner, start/end time, phase $\in \{\text{Registration, Voting, Completed}\}$), candidates with vote counters, a registration map `isRegisteredVoter[e][n]` and, per nullifier n and election e , `voteChoice[n][e]` (candidate index +1) and `voteRevisionCount[n][e]`. Election identifiers are keccak256 digests of the title, times, timestamp, sender and index. Mutating functions are restricted to the relay address (`onlyRelay`); upgrades follow a propose/approve/execute protocol with `APPROVAL_THRESHOLD = 2` of `GUARDIAN_COUNT = 3`.

The core function `castVoteRelayedV3(e, c, n, s)` checks registration, the Voting phase and the candidate index; on a first vote it increments the candidate counter; on a re-vote for a different candidate it moves one unit from the previous to the new counter (with an underflow guard). It then stores $c + 1$ and emits

$$\text{VoteCastPrivate}(e, h = \text{keccak256}(e \parallel c \parallel n \parallel s), \text{isRevote}), \quad (1)$$

where s is a fresh 32-byte salt chosen by the relay. Unlike the earlier VotingV1, which emitted `VoteCast(e, c)` and allowed exactly one ballot, the event never contains the candidate, and tallies always reflect the latest ballot.

B. Voter identity

A voter's nullifier is $n = \text{keccak256}(\text{email} \parallel \text{":"} \parallel K_{id})$, where K_{id} is the server identity secret. Observers therefore cannot map on-chain nullifiers to e-mail addresses, while the operator can recompute n to authorise a voter. Biometric session tokens are HS256 JWTs bound to n with a 10-minute lifetime. In production, all secrets and public URLs are read through a fail-closed configuration module that refuses to start when a value is missing.

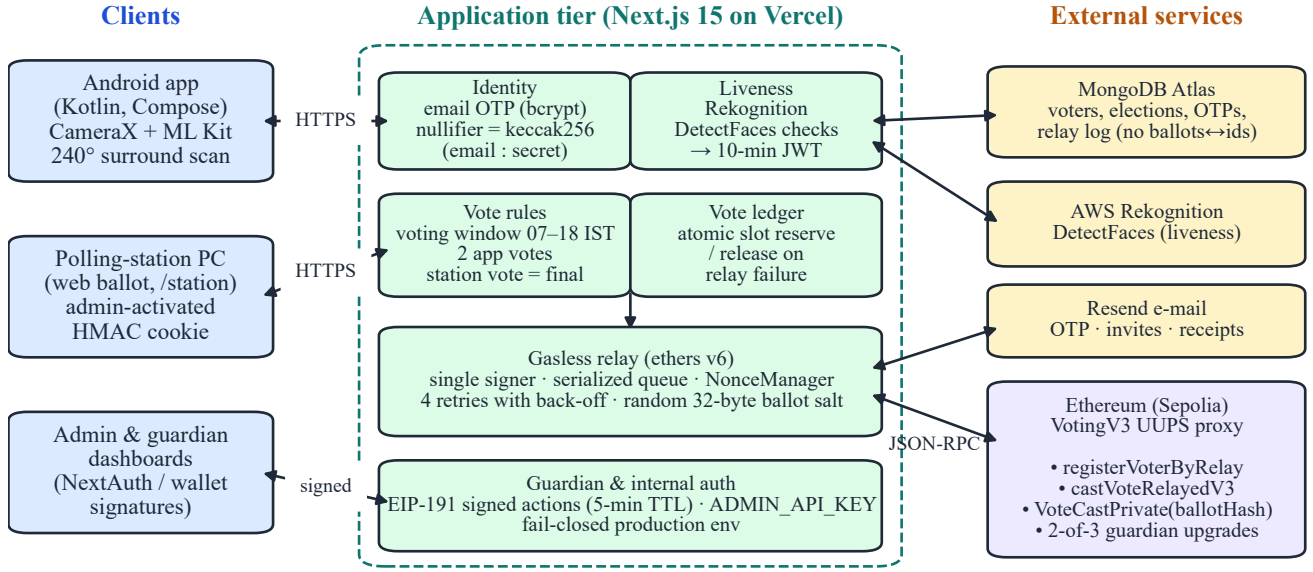


Fig. 1. Block Vote architecture. Voters never hold keys: all chain writes are relayed by the application tier. Privileged operations require either an administrator session, a guardian wallet signature, or an internal API key.

V. PROTOCOLS

A. Liveness and surround scan

On Android the front camera runs CameraX preview and analysis every 90 ms with ML Kit face detection. The voter rotates the phone while keeping the face at 10–38% of the frame height; the rotation-vector sensor accumulates yaw in a locked direction until 240° are covered, and any additional face resets the scan. After 900 ms of stable single-face framing the app auto-captures a frame and submits it. The server calls Rekognition DetectFaces and rejects the frame unless exactly one face is present with confidence ≥ 90 , brightness ≥ 30 , no eyeglasses or sunglasses detected with confidence > 80 , and eyes open. Polling stations skip the surround scan because the environment is supervised.

B. Ballot casting

Fig. 2 shows the end-to-end flow. After liveness, the voter requests a 6-digit OTP (`crypto.randomInt`, stored as a bcrypt hash, valid 5 min, at most 3 wrong attempts; 5 requests per minute per IP). `verify-otp` validates the OTP and biometric token, checks the election phase and the voting window (default 07:00–18:00 Asia/Kolkata), reserves a vote slot (Algorithm 1), and invokes the relay. The relay serialises transactions through a single queue with an ethers `NonceManager`, retries nonce errors up to four times with linear back-off, and records gas usage. On failure the slot is released. The voter receives an e-mail receipt containing the transaction hash but not the candidate; `/api/audit/verify` confirms inclusion of a transaction without revealing its content.

Algorithm 1 Atomic vote-slot reservation

Require: voter id v , channel $\in \{\text{app}, \text{station}\}$, $M = 2$

```

1:  $F \leftarrow \{\_id = v, \text{stationVoteFinal} \neq \text{true}\}$ 
2: if channel = app then
3:    $F \leftarrow F \wedge \neg(\text{votesCast} \geq M)$ 
4: end if
5:  $U \leftarrow \{\text{votesCast} += 1\}$ 
6: if channel = station then
7:    $U \leftarrow U \cup \{\text{stationVoteFinal} \leftarrow \text{true}\}$ 
8: end if
9:  $b \leftarrow \text{FINDONEANDUPDATE}(F, U, \text{return before})$ 
10: if  $b = \perp$  then return reject (VOTE_LIMIT_REACHED or STATION_VOTE_FINAL)
11: end if
12:  $r \leftarrow \text{RELAYCASTVOTE}(\cdot)$ 
13: if  $r$  failed then
14:   restore the voter document to  $b$  (release slot)
15: end if

```

C. Multi-channel ballot policy

Remote voting cannot guarantee a private booth. Following the Estonian model, Block Vote lets voters undo a coerced or mistaken remote vote, but bounds the number of remote changes to limit repeated pressure and relay cost. Fig. 3 depicts the per-voter state machine: two app ballots are allowed (the latest counts), and a ballot cast at a polling station is final and overrides any app ballot. Because VotingV3 itself permits unlimited re-votes with latest-counts semantics, the policy is enforced in the application tier by an atomic compare-and-increment on the voter document (Algorithm 1); concurrent requests cannot exceed the allowance because the filter and the increment execute as one MongoDB operation.

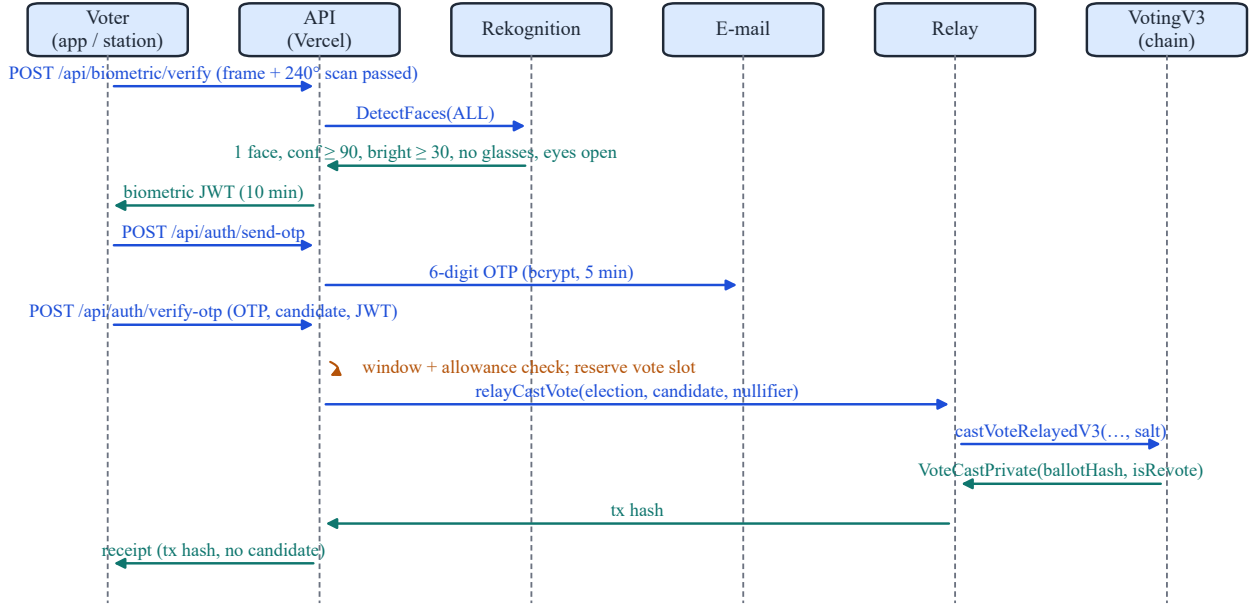


Fig. 2. Ballot-casting sequence. The candidate reaches the chain only inside contract storage; the public event carries a salted commitment.

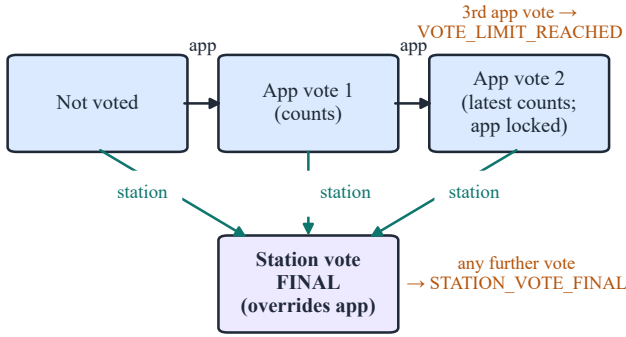


Fig. 3. Per-voter ballot state machine across the app and polling-station channels.

D. Polling-station sessions

An authenticated administrator activates a computer for one election at `/station`; the server sets an `httpOnly`, `SameSite=Strict` cookie containing a payload (station id, name, election, admin, issue and expiry time, 16h lifetime) authenticated with HMAC-SHA256 under a key derived from the station secret. The web ballot at `/station/vote` only renders when this cookie verifies, so the web channel cannot be used remotely. The kiosk resets after 2 minutes of inactivity.

E. Governance and privileged operations

Election go-live requires a guardian approval. Guardians sign an EIP-191 message binding the action, the target election and an issue time; the server recovers the signer with `ethers.verifyMessage`, requires the signature to be at most

TABLE II
TECHNOLOGY STACK

Layer	Technology (version)
Contract	Solidity 0.8.22, OpenZeppelin 5.6 (UUPS)
Chain	Hardhat 2.28 (local), Ethereum Sepolia (live)
Web / API	Next.js 15.3, React 19.1, Auth.js v5
Chain client	ethers 6.15 (NonceManager relay)
Data	MongoDB Atlas, Mongoose 9.2
Biometrics	AWS Rekognition DetectFaces; ML Kit Face 16.1.7
Messaging	Resend 6.17
Android	Kotlin 2.2, Compose, Hilt 2.57, CameraX 1.5
Hosting	Vercel (serverless functions)

five minutes old and checks the signer against the configured and on-chain guardian lists. The same mechanism protects data wiping and relay-log clearing, so knowledge of a guardian's public address alone confers no authority. Server-to-server synchronisation endpoints require an API key compared in constant time. Contract upgrades require two of three guardians on-chain.

VI. IMPLEMENTATION

Table II lists the technology stack and Table III the code base size (excluding generated code and dependencies). The web tier exposes 56 API route handlers and 11 MongoDB models. The Android app (min SDK 26, target SDK 36) supports `blockvote://` deep links and verified HTTPS App Links for invitations, and ships a production flavour restricted to HTTPS via a network security configuration.

TABLE III
CODE BASE METRICS (SOURCE LINES, SEPTEMBER 2026)

Component	Files	Lines
Solidity contracts (V1, V2, V3)	3	959
API route handlers	56	4,735
Web pages and components (JSX)	44	7,699
Server libraries (JS)	37	2,865
Android app (Kotlin)	42	5,351
Total	182	21,609

TABLE IV
AUTOMATED TEST SUITES (ALL PASSING)

Suite	Tests	Scope
VotingV3 contract	75	lifecycle, re-vote, tallies, UUPS
Security attacks (A1–A8)	30	adversarial scenarios
Production readiness	26	env, auth, links, source scan
Preflight & identity	22	eligibility cache, commitments
Vote rules	17	hours, allowance, station token
Rate limiting	13	sliding window
Biometric tokens	7	landmarks, JWT
Total	190	

VII. EVALUATION

A. Methodology

Gas was measured with a reproducible script (`scripts/benchmark-gas.js`) that deploys the UUPS proxy on the in-process Hardhat network and executes one election with 3 candidates and 50 voters, followed by 25 re-votes to a different candidate and 25 re-votes to the same candidate. To validate these numbers we exported the production relay log, which contains 114 Sepolia transactions (12–21 August 2026) with their receipts’ `gasUsed` and effective gas price. Production latency was measured from a residential client in India with 20 sequential HTTPS requests per endpoint against the live deployment. Functional correctness was assessed with the automated suites in Table IV and with an end-to-end rule test executed against a running server.

B. Automated tests

All 190 tests pass (Table IV). The adversarial suite (Table V) encodes attacks by voters, coercers, administrators, a compromised backend, contract-level manipulation, relay-level replay and guardian collusion. The production-readiness suite additionally scans the shipped source for development leftovers (test-network keys, hard-coded local URLs, demo toggles), verifies fail-closed configuration and checks guardian signatures against forgery, expiry, replay across elections and non-guardian signers. During this evaluation the suites exposed two real defects that we fixed: 35 contract assertions still expected V1 revert strings, and a production endpoint failed on cold start because a referenced data model was not registered.

End-to-end rule test. Against a running server with a seeded election, a scripted test confirmed seven properties: first app vote accepted; one change accepted; third app vote rejected (`VOTE_LIMIT_REACHED`); station vote accepted and overriding;

TABLE V
ADVERSARIAL SCENARIOS COVERED BY THE SECURITY SUITE

Adversary	Representative tests
A1 Voter	unregistered vote; zero or null-byte nullifier; direct (non-relay) cast or registration; out-of-range candidate; salt replay
A2 Coercer	no plaintext candidate in events; legacy event not emitted; coerced vote overwritten by re-vote; commitment brute-force resistance
A3 Admin	add candidate during voting; reopen completed election; single-guardian upgrade; direct counter write
A4 Backend	nullifier pre-image resistance; salt/identity correlation
A5 Contract	immutable config; underflow guard; phase machine; vote conservation
A6 Relay	revoked relay privileges; distinct salts yield distinct events
A8 Collusion	2-of-3 guardians can upgrade; one cannot

TABLE VI
GAS PER OPERATION (HARDHAT: 50 VOTERS; SEPOLIA: LIVE RELAY LOG)

Operation	V3 Hardhat mean (n)	V3 Sepolia mean (n)	V1 baseline mean
Create election	238,635 (1)	223,253 (9)	238,613
Add candidate	181,073 (3)	193,002 (17)	181,148
Register voter	55,697 (50)	55,697 (83)	55,752
Phase transition	38,619 (1)	38,619 (5)	38,585
First vote	89,946 (50)	–	66,784
Re-vote, changed	77,478 (25)	–	n/a
Re-vote, same	63,974 (25)	–	n/a

app vote after station rejected (`STATION_VOTE_FINAL`); second station vote rejected; and the on-chain `getVoteStatus` matching the off-chain ledger. A station ballot through the browser and an app ballot on a physical Android device (POCO F7) were also completed with a live face.

C. Gas consumption

Table VI and Fig. 4 report gas per operation. Registration and phase transitions match Sepolia exactly (55,697 vs. 55,697 mean; 38,619 vs. 38,619), confirming that the benchmark is representative; creation and candidate costs differ only because production strings differ in length. A first ballot costs 89,946 gas; a re-vote costs 77,478 (changed) or 63,974 (same candidate). Relative to the one-shot VotingV1 baseline (66,784 gas), privacy-preserving commitments and re-vote bookkeeping add 23,162 gas (+34.7%), dominated by one additional zero-to-non-zero storage write for `voteChoice`.

D. Operating cost

Because voters pay nothing, the operator’s marginal cost per voter is registration plus ballots. With one ballot this is $G_1 = 55,697 + 89,946 = 145,643$ gas; the worst case (two app ballots and a station override) is $G_{\max} = 145,643 + 2 \times 77,478 = 300,599$ gas. At gas price p (gwei) and N voters, the cost is $C = N G p \cdot 10^{-9}$ ETH. At the observed Sepolia median of 1.09 gwei, one voter costs 1.59×10^{-4} ETH; 10^4 voters cost 1.46 ETH at 1 gwei and 29.1 ETH at 20 gwei (Fig. 5). The 114 recorded production transactions consumed 0.01224 ETH

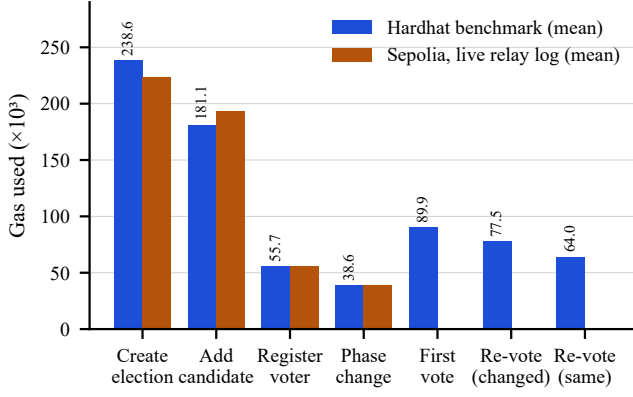


Fig. 4. Gas per operation: benchmark vs. production (Sepolia) relay transactions.

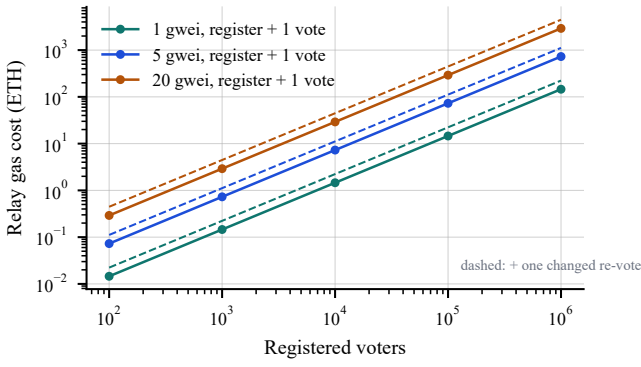


Fig. 5. Projected relay cost versus electorate size for three gas prices.

in total. These figures motivate deployment on low-fee EVM networks (layer-2 rollups or permissioned EVM chains) for large electorates, which requires no contract change.

E. Throughput

The relay waits for each receipt inside a single queue per server instance to avoid nonce collisions. Throughput per relay instance is therefore bounded by one confirmed transaction per block, i.e. about 300 ballots per hour at Ethereum’s 12 s slot time, independent of gas. A single block with a 30 M gas limit could hold $\lfloor 30 \times 10^6 / 89,946 \rfloor = 333$ first ballots, so batching or pipelined nonces could raise throughput by two orders of magnitude; we list this as future work.

F. Production latency

Fig. 6 shows round-trip latency of the live deployment. Static and light endpoints have a median of 230–260 ms; endpoints that query MongoDB and the chain have medians of 657–797 ms. The 95th-percentile outliers (1.7 s and 5.0 s) are single samples consistent with serverless cold starts. All 100 requests succeeded.

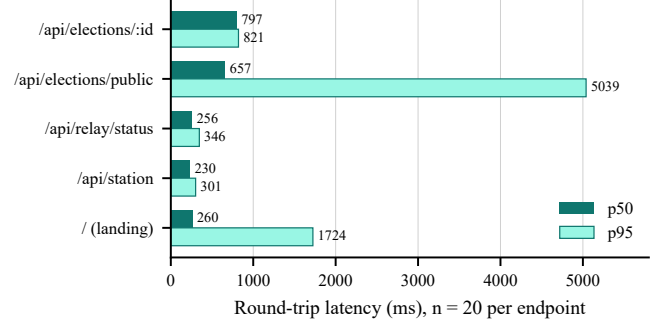


Fig. 6. Production API latency (p50 and p95, 20 requests per endpoint).

VIII. COMPARISON WITH EXISTING SYSTEMS

Table VII compares Block Vote with representative systems using properties reported in the cited literature. Block Vote’s distinguishing combination is *remote plus supervised channels with bounded re-voting* (shared only with Estonian i-Voting), a *public ledger without voter wallets or fees* (unlike the Open Vote Network), and *liveness-gated remote access*. Its main weakness relative to Helios and the Open Vote Network is ballot secrecy against the operator: the relay sees the plain-text choice and `voteChoice` is readable by anyone who can compute nullifiers.

Quantitatively, the only directly comparable measurement we report is against our own one-shot baseline (Table VI): bounded re-voting and salted commitments cost +34.7% gas per first ballot. Unlike the Open Vote Network, whose voters each submit transactions, all Block Vote transactions are issued by one relay, so per-voter on-chain cost is independent of client capabilities and hidden from the voter.

IX. SECURITY DISCUSSION AND LIMITATIONS

Relay trust. A single relay key submits all ballots; the operator could censor or reorder ballots and sees each choice. Mitigations include publishing the relay log and on-chain counts, and future use of client-side encryption with threshold decryption, or client-signed meta-transactions (e.g. EIP-2771) so that the contract verifies voter intent.

Pseudonymity, not anonymity. `voteChoice` is public per nullifier and nullifiers are emitted at registration. Secrecy against the public rests on K_{id} ; an operator or a leaked K_{id} plus the roll links ballots to voters. Replacing stored choices with homomorphic or mix-net tallying would remove this linkage at additional cost.

Coercion. Bounded re-voting and the polling-station override are operational, not cryptographic, defences [10]; a coercer who controls a voter until polls close is not defeated.

Liveness versus identity. The server checks liveness and image quality but does not match the face against an enrolled reference, so an enrolled voter could let another live person vote after completing OTP. Enrolment-time face matching or ISO/IEC 30107-3 evaluated presentation-attack detection [17] would strengthen this.

TABLE VII
QUALITATIVE COMPARISON WITH EXISTING VOTING SYSTEMS (✓ PROVIDED, ◦ PARTIAL OR CONDITIONAL, ✗ NOT PROVIDED)

Property	EVM + VVPAT [4]	Estonia i-Voting [5], [6]	Helios [7]	Voatz [14]	Open Vote Net. [11]	Block Vote
Remote voting	✗	✓	✓	✓	✓	✓
Supervised in-person channel	✓	✓	✗	✗	✗	✓
Re-voting / in-person override	✗	✓	◦	✗	✗	✓ (bounded)
Public tamper-evident record	✗	✗	✓ (bulletin board)	◦ (permissioned)	✓	✓ (Ethereum)
Universally verifiable tally	◦ (paper audit)	✗	✓	✗	✓	◦ (on-chain counters)
Ballot secrecy vs. public	✓	✓	✓	◦	✓	✓ (commitments)
Ballot secrecy vs. operator	✓	◦ (key holders)	◦ (trustees)	✗	✓	✗ (relay sees choice)
Automated liveness / biometrics	✗ (manual ID)	✗	✗	✓	✗	✓ (liveness only)
No voter wallet or fees	✓	✓	✓	✓	✗	✓
Contract/system upgrade governance	n/a	n/a	n/a	n/a	✗	✓ (2-of-3 UUPS)
Demonstrated scale	national	national	organisational	pilots	boardroom	prototype

Operational gaps. Go-live approval requires one guardian (contract upgrades require two) and upgrades have no time-lock; request rate limiting is kept in per-instance memory, which is weak on serverless platforms; relay throughput is limited per instance; and the Android client has minimal automated tests. The production gas-analytics endpoint is publicly readable, which exposes operational metadata.

X. CONCLUSION AND FUTURE WORK

Block Vote shows that a public-chain voting back end can be made invisible to voters: no wallets, no fees, a native mobile app, and a supervised fallback, while every ballot and tally update remains an auditable on-chain transaction. The multi-channel policy (two remote ballots, final polling-station override) offers a practical, low-cost coercion mitigation, and our measurements show a per-voter cost of 145,643 gas with production API medians below 0.8s. Future work targets (i) client-side ballot encryption with threshold tallying to remove operator visibility, (ii) pipelined or batched relaying and layer-2 deployment for throughput and cost, (iii) enrolment-time face matching, and (iv) a shared rate-limit store and timelocked upgrades.

ARTIFACT AVAILABILITY

The repository contains the gas benchmark (Voting/scripts/benchmark-gas.js), all test suites, and the measurement data and figure scripts used in this paper (docs/paper/).

REFERENCES

- [1] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [2] G. Wood, “Ethereum: A secure decentralised generalised transaction ledger,” Ethereum Yellow Paper, 2014.
- [3] S. Park, M. Specter, N. Narula, and R. L. Rivest, “Going from bad to worse: From Internet voting to blockchain voting,” *J. Cybersecurity*, vol. 7, no. 1, 2021.
- [4] Election Commission of India, “Status paper on Electronic Voting Machine (EVM),” New Delhi, India.
- [5] Ü. Madise and T. Martens, “E-voting in Estonia 2005: The first practice of country-wide binding Internet voting in the world,” in *Proc. Electronic Voting (EVOTE)*, GI LNI P-86, 2006, pp. 15–26.
- [6] D. Springall *et al.*, “Security analysis of the Estonian Internet voting system,” in *Proc. ACM CCS*, 2014, pp. 703–715.
- [7] B. Adida, “Helios: Web-based open-audit voting,” in *Proc. 17th USENIX Security Symp.*, 2008, pp. 335–348.
- [8] J. Benaloh, “Simple verifiable elections,” in *Proc. USENIX/ACCURATE Electronic Voting Technology Workshop (EVT)*, 2006.
- [9] D. Chaum, “Secret-ballot receipts: True voter-verifiable elections,” *IEEE Security & Privacy*, vol. 2, no. 1, pp. 38–47, 2004.
- [10] A. Juels, D. Catalano, and M. Jakobsson, “Coercion-resistant electronic elections,” in *Proc. ACM Workshop on Privacy in the Electronic Society (WPES)*, 2005, pp. 61–70.
- [11] P. McCorry, S. F. Shahandashti, and F. Hao, “A smart contract for boardroom voting with maximum voter privacy,” in *Financial Cryptography and Data Security (FC)*, LNCS 10322, 2017, pp. 357–375.
- [12] N. Kshetri and J. Voas, “Blockchain-enabled e-voting,” *IEEE Software*, vol. 35, no. 4, pp. 95–99, 2018.
- [13] F. P. Hjälmarsen, G. K. Hreiðarsson, M. Hamdaqa, and G. Hjälmtysson, “Blockchain-based e-voting system,” in *Proc. IEEE 11th Int. Conf. Cloud Computing (CLOUD)*, 2018, pp. 983–986.
- [14] M. A. Specter, J. Koppel, and D. Weitzner, “The ballot is busted before the blockchain: A security analysis of Voatz, the first Internet voting application used in U.S. federal elections,” in *Proc. 29th USENIX Security Symp.*, 2020, pp. 1535–1553.
- [15] G. Barros and P. Gallagher, “EIP-1822: Universal Upgradeable Proxy Standard (UUPS),” Ethereum Improvement Proposals, 2019.
- [16] M. Swende and N. Johnson, “EIP-191: Signed data standard,” Ethereum Improvement Proposals, 2016.
- [17] ISO/IEC 30107-3:2017, *Information technology—Biometric presentation attack detection—Part 3: Testing and reporting*.
- [18] P. A. Grassi *et al.*, “Digital identity guidelines: Authentication and lifecycle management,” NIST SP 800-63B, 2017.